



Actividad de Formación Práctica 3

Funciones no bloqueantes, aplicación con SysTick en STM32CubeIDE, programación de microcontroladores.

Integrantes:

Lucero Dario

Pistan Ulises

Leguizamon Marcelo

Rivero Andres Martin

Introducción

Las funciones de retardo no bloqueantes permiten pausar una tarea sin detener todo el flujo de ejecución de un programa. A diferencia de las funciones bloqueantes, que detienen el hilo principal hasta que se completa el retardo, las no bloqueantes permiten continuar ejecutando otras tareas en paralelo. Trabajar de esta forma tiene varias ventajas:

- 1- Mejor rendimiento: ya que permite la multitarea simultáneamente, podemos atender varias solicitudes sin que el sistema se detenga.
- 2- Escalabilidad: al no bloquear el flujo de ejecución, es posible agregar tareas sin necesidad de aumentar los recursos.
- 3- Experiencia fluida en la ejecución de las tareas: evitamos que la lectura de datos, o de pines se vea bloqueada en algún instante.

Creación de la librería API_Delay

Header

En el encabezado de la librería seguimos los siguientes pasos:

- 1- Incluimos las librerías `<stdint.h>` y `<stdbool.h>`.
- 2- Creamos el typedef `"tick_t"` (enteros de 32 bits que guardan el tiempo transcurrido en milisegundos)
- 3- Creamos el typedef `"bool_t"` (variables booleanas, que guardan true o false).
- 4- Creamos la estructura `delay_t`
- 5- Declaramos las funciones: `delayInit`, `delayRead`, `delayWrite`.

```
/*
 * API_Delay.h
 *
 * Created on: August 31, 2025
 * Author: Grupo 1_TD2 Pistan Ulises, Lucero Dario Alejandro, Rivero Martin, Leguizamon Marcelo
 */

#ifndef API_INC_API_DELAY_H_
#define API_INC_API_DELAY_H_

/* Includes */
#include <stdint.h>
#include <stdbool.h>

/* Typedefs */
typedef uint32_t tick_t;
typedef bool bool_t;
typedef struct{
    tick_t startTime;
    tick_t duration;
    bool_t running;
} delay_t;

/* Declaracion de Funciones */
void delayInit(delay_t* delay, tick_t duration);
bool_t delayRead(delay_t* delay);
void delayWrite(delay_t* delay, tick_t duration);

#endif /* API_INC_API_DELAY_H_ */
```

Source

En el archivo de extensión “.c”, implementamos las funciones que declaramos en la cabecera.

delayInit

En esta función inicializamos el delay, recibimos de parámetro un tiempo de duración el cual le asignamos mediante el atributo *duration* de la estructura *delay_t*. Luego iniciamos *running* en *false*, esto se hace porque el inicio del conteo se hace al llamar a otra función llamada “*delayRead*”.

delayRead

Esta función revisa si ya paso el tiempo correspondiente al delay previamente definido, esto lo hace a través del SysTick, con la función “*HAL_GetTick()*” que devuelve el tiempo transcurrido en milisegundos desde que inicio la aplicación en el microcontrolador.

Si *running* es *false*, lo que hace es iniciar el conteo, almacenando en la variable *starTime* el tiempo en el que comienza el conteo, luego establece *running* en *true*, ya que el contador está corriendo. Si *running* es *true*, esto significa que el delay está en ejecución por lo tanto a cada llamado de la función corrobora si el tiempo de duración se ha cumplido, de ser así retorna *true* a la función informando que el delay ha finalizado y establece *running* en *false* ya que el delay dejo de contar. Si el tiempo de duración todavía no se cumplió retorna *false* a la función.

delayWrite

Lo único que hace esta función es permitir sobrescribir el tiempo de duración del delay.

```
+ * API_Delay.h
#include "main.h"
#include "API_Delay.h"

+ * @brief Inicializacion del delay, establece la duracion
- void delayInit(delay_t * delay, tick_t duration){
    delay->duration = duration;
    delay->running = false;
}

+ * @brief Revisa si ya paso el tiempo correspondiente al delay
- bool_t delayRead(delay_t* delay){
    if(delay->running){
        if ((HAL_GetTick() - delay->starTime) >= delay->duration){
            delay->running = false;
            return true;
        }else{
            return false;
        }
    }else{
        delay->starTime = HAL_GetTick();
        delay->running = true;
        return false;
    }
}

+ * @brief Reescribe el tiempo de duracion
- void delayWrite(delay_t* delay, tick_t duration){
    delay->duration = duration;
}
```

ENLACES:

REPOSITORIO GRUPAL: https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025.git

APP 3 1: https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/main/AFP_3_TDII_2025/App_3_1_Grupo_1_2025

APP 3 2: https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/main/AFP_3_TDII_2025/App_3_2_Grupo_1_2025

APP 3 3: https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/main/AFP_3_TDII_2025/App_3_3_Grupo_%201_2025

APP 3 4: https://github.com/Dariuss-ing-electronica/Grupo_1_TDII_2025/tree/main/AFP_3_TDII_2025/App_3_4_Grupo_1_2025